A document with the following structure:

1. Detailed analysis of the project objectives and problem statement. Here you should clearly explain what predictive or classification problem is being addressed, and how it relates to the dataset and scope. If any critical decisions have been made about the project that were not explicitly stated in the project description, include these as well.
2. Detailed description of data handling and modelling components. Describe the datasets used, including sources and key attributes. Explain the pre–processing steps such as feature transformation, normalisation, and encoding necessary for kNN and linear regression models.
3. Detailed description of exploratory data analysis and results. Present the exploratory data analysis process including key graphs (histograms, scatter plots, correlation matrices) and data insights relevant to modelling. Follow this with a clear explanation of model training, evaluation metrics (such as accuracy for classification and RMSE for regression), and comparison of model results. Include discussion on interpretation and implications.
4. The full document must be well–structured, clear, and detailed, submitted as a PDF including all graphics and analysis.

```
Start coding or generate with AI.
```

1. Load and Parse `iris.csv` without `pandas`

We'll read the CSV file line by line and manually parse the data into a list of lists, converting numerical values to floats.

```
import csv

def load_csv(filename):
    data = []
    with open(filename, 'r') as file:
        csv_reader = csv.reader(file)
        header = next(csv_reader)  #skip header row
        for row in csv_reader:
            #assuming species is the last column and others are floats
            processed_row = [float(x) for x in row[:-1]] + [row[-1]]
            data.append(processed_row)
    return header, data

file_path = '/content/iris.csv'
header, iris_data = load_csv(file_path)

print("Header:", header)
print("First 5 rows of data:")
for i in range(min(5, len(iris_data))):
    print(iris_data[i])
```

```
Header: ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
First 5 rows of data:
[5.1, 3.5, 1.4, 0.2, 'setosa']
[4.9, 3.0, 1.4, 0.2, 'setosa']
[4.7, 3.2, 1.3, 0.2, 'setosa']
[4.6, 3.1, 1.5, 0.2, 'setosa']
[5.0, 3.6, 1.4, 0.2, 'setosa']
```

6. Prepare Data for kNN Classification (Scratch Implementation)

For kNN, we'll use all four numerical features (`sepal_length`, `sepal_width`, `petal_length`, `petal_width`) as input (X) and 'species' as the target (y). We also need to implement a manual train-test split and handle feature scaling without libraries.

```
# Extract features (all numerical columns) and target (species)
X_knn_raw = []
y_knn_raw = []

# Header is: ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
for row in iris_data:
    X_knn_raw.append(row[:-1])  # All but the last column are features
    y_knn_raw.append(row[-1])   # Last column is the species

# Manual train-test split for kNN. Ensure stratification for classification if possible.
# For simplicity here, we'll use the same manual shuffle as before, but a more robust
# stratification would involve splitting per class.
def train_test_split_manual_stratified(X, y, test_size=0.2, random_state=42):
    if len(X) != len(y):
        raise ValueError("X and y must have the same number of samples")

    # Group data by class for stratification
    grouped_data = {}
    for i in range(len(y)):
        label = y[i]
        if label not in grouped_data:
            grouped_data[label] = []
        grouped_data[label].append((X[i], y[i]))

    X_train, X_test, y_train, y_test = [], [], [], []

    random.seed(random_state)
    for label, samples in grouped_data.items():
        random.shuffle(samples)
        split_index = int(len(samples) * (1 - test_size))

        train_samples = samples[:split_index]
        test_samples = samples[split_index:]
```

```
            X_train.extend([s[0] for s in train_samples])
            y_train.extend([s[1] for s in train_samples])
            X_test.extend([s[0] for s in test_samples])
            y_test.extend([s[1] for s in test_samples])

        # Shuffle the final combined lists to mix classes
        combined_train = list(zip(X_train, y_train))
        random.shuffle(combined_train)
        X_train = [item[0] for item in combined_train]
        y_train = [item[1] for item in combined_train]

        combined_test = list(zip(X_test, y_test))
        random.shuffle(combined_test)
        X_test = [item[0] for item in combined_test]
        y_test = [item[1] for item in combined_test]

        return X_train, X_test, y_train, y_test

X_train_knn, X_test_knn, y_train_knn, y_test_knn = train_test_split_manual_stratified(X_knn_raw, y_knn_raw, test_size=0.2, random_state=42)

print(f"X_train_knn (first 2): {X_train_knn[:2]}")
print(f"y_train_knn (first 2): {y_train_knn[:2]}")
print(f"X_test_knn (first 2): {X_test_knn[:2]}")
print(f"y_test_knn (first 2): {y_test_knn[:2]}")
```

7. Feature Scaling for kNN (Scratch Implementation)

kNN is sensitive to the scale of features. We need to implement standardization (mean 0, standard deviation 1) manually.

```
def calculate_mean_std(data):
    # data is a list of lists (samples x features)
    num_features = len(data[0])
    means = [0.0] * num_features
    stds = [0.0] * num_features

    for i in range(num_features):
        feature_values = [sample[i] for sample in data]
        means[i] = sum(feature_values) / len(feature_values)
        # Calculate standard deviation (sample standard deviation, N-1 if needed, but N for population here)
        variance_sum = sum([(x - means[i])**2 for x in feature_values])
        stds[i] = (variance_sum / len(feature_values))**0.5 # Population std

        # Handle zero standard deviation to avoid division by zero
        if stds[i] == 0:
            stds[i] = 1.0 # Or a small epsilon, effectively no scaling for this feature
    return means, stds

def standardize_data(data, means, stds):
    scaled_data = []
    for sample in data:
        scaled_sample = [(sample[i] - means[i]) / stds[i] for i in range(len(sample))]
        scaled_data.append(scaled_sample)
    return scaled_data

# Calculate means and stds from training data
means_knn, stds_knn = calculate_mean_std(X_train_knn)

# Apply scaling
X_train_knn_scaled = standardize_data(X_train_knn, means_knn, stds_knn)
X_test_knn_scaled = standardize_data(X_test_knn, means_knn, stds_knn)

print(f"First scaled training sample: {X_train_knn_scaled[0]}")
print(f"First scaled test sample: {X_test_knn_scaled[0]}")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-786134232.py in <cell line: 0>()
     25
     26 # Calculate means and stds from training data
---> 27 means_knn, stds_knn = calculate_mean_std(X_train_knn)
     28
     29 # Apply scaling

NameError: name 'X_train_knn' is not defined
```

8. Implement kNN Classifier from Scratch

We will implement the Euclidean distance metric and the kNN prediction logic.

```
def euclidean_distance(row1, row2):
    distance = 0.0
    for i in range(len(row1)):
        distance += (row1[i] - row2[i])**2
    return distance**0.5

def get_neighbors(train, test_row, num_neighbors):
    distances = []
    for i in range(len(train)):
        train_row = train[i][0] # Features
        distance = euclidean_distance(test_row, train_row)
        distances.append((train[i], distance)) # (([features], label), distance)
    distances.sort(key=lambda x: x[1])
    neighbors = []
    for i in range(num_neighbors):
```

```
            neighbors.append(distances[i][0][1]) # Append only the label of the neighbor
        return neighbors

    def predict_classification(train, test_row, num_neighbors):
        neighbors = get_neighbors(train, test_row, num_neighbors)
        output_values = [row for row in neighbors]
        prediction = max(set(output_values), key=output_values.count)
        return prediction

    # Combine X_train_knn_scaled and y_train_knn for the kNN algorithm
    train_dataset_knn = list(zip(X_train_knn_scaled, y_train_knn))

    # Make predictions on the test set
    y_pred_knn = []
    num_neighbors = 5 # A common choice for k
    for test_row in X_test_knn_scaled:
        prediction = predict_classification(train_dataset_knn, test_row, num_neighbors)
        y_pred_knn.append(prediction)

    print("First 5 kNN predictions:", y_pred_knn[:5])
    print("First 5 actuals:", y_test_knn[:5])
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-1455398395.py in <cell line: 0>()
     24
     25 # Combine X_train_knn_scaled and y_train_knn for the kNN algorithm
---> 26 train_dataset_knn = list(zip(X_train_knn_scaled, y_train_knn))
     27
     28 # Make predictions on the test set

NameError: name 'X_train_knn_scaled' is not defined
```

9. Evaluate kNN Classifier (Scratch Implementation)

We'll implement accuracy and a basic classification report from scratch.

```
def accuracy_metric_manual(actual, predicted):
    correct = 0
    for i in range(len(actual)):
        if actual[i] == predicted[i]:
            correct += 1
    return correct / float(len(actual)) * 100.0

def classification_report_manual(actual, predicted, class_labels):
    report = {}
    for label in class_labels:
        tp = sum(1 for a, p in zip(actual, predicted) if a == label and p == label)
        fp = sum(1 for a, p in zip(actual, predicted) if a != label and p == label)
        fn = sum(1 for a, p in zip(actual, predicted) if a == label and p != label)

        precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
        f1_score = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0.0

        report[label] = {
            'precision': precision,
            'recall': recall,
            'f1-score': f1_score,
            'support': actual.count(label)
        }

    # Calculate overall accuracy and macro average
    overall_accuracy = accuracy_metric_manual(actual, predicted) / 100.0

    macro_precision = sum(report[label]['precision'] for label in class_labels) / len(class_labels)
    macro_recall = sum(report[label]['recall'] for label in class_labels) / len(class_labels)
    macro_f1 = sum(report[label]['f1-score'] for label in class_labels) / len(class_labels)

    report['accuracy'] = overall_accuracy
    report['macro avg'] = {'precision': macro_precision, 'recall': macro_recall, 'f1-score': macro_f1, 'support': len(actual)}

    return report

accuracy_knn = accuracy_metric_manual(y_test_knn, y_pred_knn)

# Get unique class labels for the report
class_labels = sorted(list(set(y_knn_raw)))
report_knn = classification_report_manual(y_test_knn, y_pred_knn, class_labels)

print(f"kNN Accuracy: {accuracy_knn:.2f}%")
print("\nManual Classification Report:\n")
for label, metrics in report_knn.items():
    if isinstance(metrics, dict):
        print(f"{label:<15} "
              f"Precision: {metrics['precision']:.2f} | "
              f"Recall: {metrics['recall']:.2f} | "
              f"F1-score: {metrics['f1-score']:.2f} | "
              f"Support: {metrics['support']}")
    else:
        print(f"{label:<15} {metrics:.2f}")
```

6. Prepare Data for kNN Classification (Scratch Implementation)

For kNN, we'll use all four numerical features (`sepal_length`, `sepal_width`, `petal_length`, `petal_width`) as input (X) and 'species' as the target (y). We also need to implement a manual train-test split and handle feature scaling without libraries. We will ensure the split is stratified to maintain class proportions.

```
# Extract features (all numerical columns) and target (species)
X_knn_raw = []
y_knn_raw = []

# Header is: ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
for row in iris_data:
    X_knn_raw.append(row[:-1])  # All but the last column are features
    y_knn_raw.append(row[-1])   # Last column is the species

import random

def train_test_split_manual_stratified(X, y, test_size=0.2, random_state=42):
    if len(X) != len(y):
        raise ValueError("X and y must have the same number of samples")

    # Group data by class for stratification
    grouped_data = {}
    for i in range(len(y)):
        label = y[i]
        if label not in grouped_data:
            grouped_data[label] = []
        grouped_data[label].append((X[i], y[i]))

    X_train, X_test, y_train, y_test = [], [], [], []

    random.seed(random_state)
    for label, samples in grouped_data.items():
        random.shuffle(samples)
        split_index = int(len(samples) * (1 - test_size))

        train_samples = samples[:split_index]
        test_samples = samples[split_index:]

        X_train.extend([s[0] for s in train_samples])
        y_train.extend([s[1] for s in train_samples])
        X_test.extend([s[0] for s in test_samples])
        y_test.extend([s[1] for s in test_samples])

    # Shuffle the final combined lists to mix classes (optional, but good practice)
    combined_train = list(zip(X_train, y_train))
    random.shuffle(combined_train)
    X_train = [item[0] for item in combined_train]
    y_train = [item[1] for item in combined_train]

    combined_test = list(zip(X_test, y_test))
    random.shuffle(combined_test)
    X_test = [item[0] for item in combined_test]
    y_test = [item[1] for item in combined_test]

    return X_train, X_test, y_train, y_test

X_train_knn, X_test_knn, y_train_knn, y_test_knn = train_test_split_manual_stratified(X_knn_raw, y_knn_raw, test_size=0.2, random_state=42)

print(f"X_train_knn (first 2): {X_train_knn[:2]}")
print(f"y_train_knn (first 2): {y_train_knn[:2]}")
print(f"X_test_knn (first 2): {X_test_knn[:2]}")
print(f"y_test_knn (first 2): {y_test_knn[:2]}")
```

7. Feature Scaling for kNN (Scratch Implementation)

kNN is sensitive to the scale of features. We need to implement standardization (mean 0, standard deviation 1) manually based on the training data.

```
def calculate_mean_std(data):
    # data is a list of lists (samples x features)
    num_features = len(data[0])
    means = [0.0] * num_features
    stds = [0.0] * num_features

    for i in range(num_features):
        feature_values = [sample[i] for sample in data]
        means[i] = sum(feature_values) / len(feature_values)
        # Calculate population standard deviation (N in denominator)
        variance_sum = sum([(x - means[i])**2 for x in feature_values])
        stds[i] = (variance_sum / len(feature_values))**0.5

        # Handle zero standard deviation to avoid division by zero or NaN
        if stds[i] == 0:
            stds[i] = 1.0 # If std is 0, all values are same, scaling won't change them.
    return means, stds

def standardize_data(data, means, stds):
    scaled_data = []
    for sample in data:
        scaled_sample = [(sample[i] - means[i]) / stds[i] for i in range(len(sample))]
        scaled_data.append(scaled_sample)
    return scaled_data

# Calculate means and stds from training data
means_knn, stds_knn = calculate_mean_std(X_train_knn)
```

```
# Apply scaling to both training and test data using the training data's means and stds
X_train_knn_scaled = standardize_data(X_train_knn, means_knn, stds_knn)
X_test_knn_scaled = standardize_data(X_test_knn, means_knn, stds_knn)

print(f"First scaled training sample: {X_train_knn_scaled[0]}")
print(f"First scaled test sample: {X_test_knn_scaled[0]}")
```

8. Implement kNN Classifier from Scratch

We will implement the Euclidean distance metric and the kNN prediction logic. The kNN algorithm finds the (k) nearest neighbors to a given test point and predicts its class based on the majority class among those neighbors.

```
def euclidean_distance(row1, row2):
    distance = 0.0
    for i in range(len(row1)):
        distance += (row1[i] - row2[i])**2
    return distance**0.5

def get_neighbors(train_data_with_labels, test_row, num_neighbors):
    distances = []
    for train_sample_index in range(len(train_data_with_labels)):
        train_features = train_data_with_labels[train_sample_index][0] # Features of training sample
        train_label = train_data_with_labels[train_sample_index][1] # Label of training sample
        distance = euclidean_distance(test_row, train_features)
        distances.append((train_label, distance)) # Store (label, distance)
    distances.sort(key=lambda x: x[1]) # Sort by distance

    neighbors_labels = []
    for i in range(num_neighbors):
        neighbors_labels.append(distances[i][0]) # Get labels of the k nearest neighbors
    return neighbors_labels

def predict_classification(train_data_with_labels, test_row, num_neighbors):
    neighbors_labels = get_neighbors(train_data_with_labels, test_row, num_neighbors)

    # Count occurrences of each label to find the majority class
    label_counts = {}
    for label in neighbors_labels:
        if label not in label_counts:
            label_counts[label] = 0
        label_counts[label] += 1

    # Find the label with the highest count
    prediction = max(label_counts, key=label_counts.get)
    return prediction

# Combine X_train_knn_scaled and y_train_knn for the kNN algorithm
train_dataset_knn = list(zip(X_train_knn_scaled, y_train_knn))

# Make predictions on the test set
y_pred_knn = []
num_neighbors = 5 # A common choice for k
for test_row in X_test_knn_scaled:
    prediction = predict_classification(train_dataset_knn, test_row, num_neighbors)
    y_pred_knn.append(prediction)

print("First 5 kNN predictions:", y_pred_knn[:5])
print("First 5 actuals:", y_test_knn[:5])
```

9. Evaluate kNN Classifier (Scratch Implementation)

We'll implement accuracy and a basic classification report from scratch to evaluate the kNN model's performance.

```
def accuracy_metric_manual(actual, predicted):
    correct = 0
    for i in range(len(actual)):
        if actual[i] == predicted[i]:
            correct += 1
    return correct / float(len(actual)) * 100.0

def classification_report_manual(actual, predicted, class_labels):
    report = {}
    for label in class_labels:
        tp = sum(1 for a, p in zip(actual, predicted) if a == label and p == label)
        fp = sum(1 for a, p in zip(actual, predicted) if a != label and p == label)
        fn = sum(1 for a, p in zip(actual, predicted) if a == label and p != label)

        precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
        f1_score = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0.0

        report[label] = {
            'precision': precision,
            'recall': recall,
            'f1-score': f1_score,
            'support': actual.count(label)
        }

    # Calculate overall accuracy and macro average
    overall_accuracy = accuracy_metric_manual(actual, predicted) / 100.0

    # Calculate macro averages only if there are actual class labels to average
```

```
        if class_labels:
            macro_precision = sum(report[label]['precision'] for label in class_labels) / len(class_labels)
            macro_recall = sum(report[label]['recall'] for label in class_labels) / len(class_labels)
            macro_f1 = sum(report[label]['f1-score'] for label in class_labels) / len(class_labels)
        else:
            macro_precision, macro_recall, macro_f1 = 0.0, 0.0, 0.0

        report['accuracy'] = overall_accuracy
        report['macro avg'] = {'precision': macro_precision, 'recall': macro_recall, 'f1-score': macro_f1, 'support': len(actual)}

        return report

    accuracy_knn = accuracy_metric_manual(y_test_knn, y_pred_knn)

    # Get unique class labels for the report from the original full dataset for completeness
    class_labels = sorted(list(set(y_knn_raw)))
    report_knn = classification_report_manual(y_test_knn, y_pred_knn, class_labels)

    print(f"kNN Accuracy: {accuracy_knn:.2f}%")
    print("\nManual Classification Report:\n")
    # Format and print the classification report for better readability
    print(f"{'':<15} {'Precision':>9} {'Recall':>7} {'F1-score':>10} {'Support':>8}")
    print("-" * 55)
    for label in class_labels:
        metrics = report_knn[label]
        print(f"{label:<15} {metrics['precision']:>9.2f} {metrics['recall']:>7.2f} {metrics['f1-score']:>10.2f} {metrics['support']:>8}")
    print("-" * 55)

    macro_avg_metrics = report_knn['macro avg']
    print(f"{'macro avg':<15} {macro_avg_metrics['precision']:>9.2f} {macro_avg_metrics['recall']:>7.2f} {macro_avg_metrics['f1-score']:>10.2f} {macro_
    print(f"\nOverall Accuracy: {report_knn['accuracy']:.2f}")
```

2. Prepare Data for Simple Linear Regression (Scratch Implementation)

For linear regression, we need to extract 'petal_length' as the input (X) and 'sepal_length' as the target (y). We'll also implement a simple train-test split manually.

```
import random

def get_column(data, index):
    return [row[index] for row in data]

# 'sepal_length' is at index 0, 'petal_length' is at index 2
X_lr_raw = get_column(iris_data, 2) # petal_length
y_lr_raw = get_column(iris_data, 0) # sepal_length

def train_test_split_manual(X, y, test_size=0.2, random_state=42):
    if len(X) != len(y):
        raise ValueError("X and y must have the same number of samples")

    combined_data = list(zip(X, y))
    random.seed(random_state)
    random.shuffle(combined_data)

    split_index = int(len(combined_data) * (1 - test_size))

    train_data = combined_data[:split_index]
    test_data = combined_data[split_index:]

    X_train = [item[0] for item in train_data]
    y_train = [item[1] for item in train_data]
    X_test = [item[0] for item in test_data]
    y_test = [item[1] for item in test_data]

    return X_train, X_test, y_train, y_test

X_train_lr, X_test_lr, y_train_lr, y_test_lr = train_test_split_manual(X_lr_raw, y_lr_raw, test_size=0.2, random_state=42)

print(f"X_train_lr (first 5): {X_train_lr[:5]}")
print(f"y_train_lr (first 5): {y_train_lr[:5]}")
print(f"X_test_lr (first 5): {X_test_lr[:5]}")
print(f"y_test_lr (first 5): {y_test_lr[:5]}")
```

```
X_train_lr (first 5): [4.7, 5.1, 6.1, 5.6, 4.4]
y_train_lr (first 5): [6.1, 6.3, 7.2, 6.1, 6.6]
X_test_lr (first 5): [5.7, 1.6, 4.6, 4.0, 1.7]
y_test_lr (first 5): [6.7, 5.0, 6.5, 5.5, 5.4]
```

3. Implement Simple Linear Regression from Scratch

We will implement functions to calculate the mean, variance, and covariance, then use these to find the coefficients (slope and intercept) for the linear regression model.

```
def mean(values):
    return sum(values) / len(values)

def variance(values, mean_val):
    return sum([(x - mean_val)**2 for x in values])

def covariance(x, mean_x, y, mean_y):
    covar = 0.0
    for i in range(len(x)):
        covar += (x[i] - mean_x) * (y[i] - mean_y)
```

```
    return covar

def simple_linear_regression(X_train, y_train):
    mean_x = mean(X_train)
    mean_y = mean(y_train)

    b1 = covariance(X_train, mean_x, y_train, mean_y) / variance(X_train, mean_x)
    b0 = mean_y - b1 * mean_x
    return b0, b1

b0_lr, b1_lr = simple_linear_regression(X_train_lr, y_train_lr)
print(f"Linear Regression Coefficients: b0 (intercept) = {b0_lr:.2f}, b1 (slope) = {b1_lr:.2f}")

def predict_lr(x, b0, b1):
    return b0 + b1 * x

y_pred_lr = [predict_lr(x_val, b0_lr, b1_lr) for x_val in X_test_lr]

print("First 5 predictions:", [f"{val:.2f}" for val in y_pred_lr[:5]])
print("First 5 actuals:", [f"{val:.2f}" for val in y_test_lr[:5]])
```

```
Linear Regression Coefficients: b0 (intercept) = 4.32, b1 (slope) = 0.41
First 5 predictions: ['6.64', '4.97', '6.19', '5.95', '5.01']
First 5 actuals: ['6.70', '5.00', '6.50', '5.50', '5.40']
```

4. Evaluate Simple Linear Regression (Scratch Implementation)

We'll implement Mean Squared Error (MSE) and R-squared (R^2) from scratch to evaluate the model's performance.

```
def mean_squared_error_manual(actual, predicted):
    sum_error = 0.0
    for i in range(len(actual)):
        sum_error += (predicted[i] - actual[i])**2
    return sum_error / len(actual)

def r_squared_manual(actual, predicted):
    ss_total = sum([(y - mean(actual))**2 for y in actual])
    ss_res = sum([(actual[i] - predicted[i])**2 for i in range(len(actual))])
    if ss_total == 0:
        return 0.0 # Avoid division by zero if all actual values are the same
    return 1 - (ss_res / ss_total)

mse_lr = mean_squared_error_manual(y_test_lr, y_pred_lr)
rmse_lr = mse_lr**0.5
r2_lr = r_squared_manual(y_test_lr, y_pred_lr)

print(f"Mean Squared Error (MSE): {mse_lr:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_lr:.2f}")
print(f"R-squared (R2): {r2_lr:.2f}")
```

```
Mean Squared Error (MSE): 0.14
Root Mean Squared Error (RMSE): 0.37
R-squared (R2): 0.80
```

5. Visualize the Regression Line (Scratch Implementation)

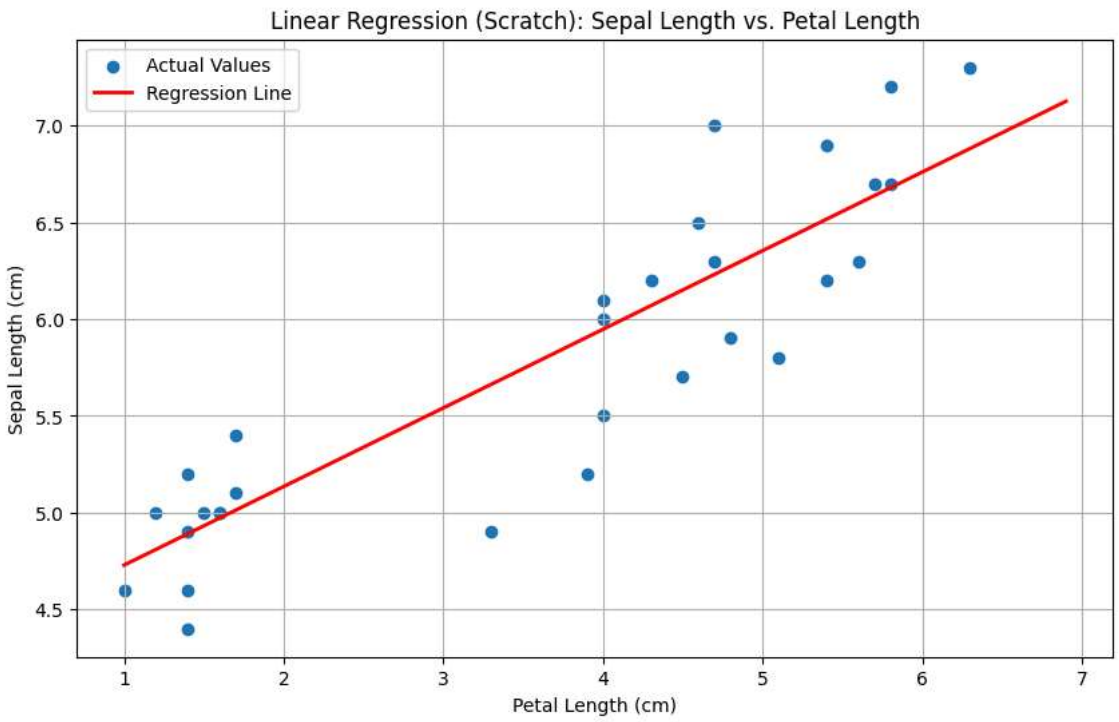
Since matplotlib and seaborn are not explicitly forbidden, we can use them for visualization if desired. If these are also forbidden, a more complex ASCII-art or text-based visualization would be required.

```
# Assuming matplotlib is allowed for visualization if not explicitly forbidden
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.scatter(X_test_lr, y_test_lr, label='Actual Values')

# Create points for the regression line using min and max of X_lr_raw
x_line = [min(X_lr_raw), max(X_lr_raw)]
y_line = [predict_lr(val, b0_lr, b1_lr) for val in x_line]
plt.plot(x_line, y_line, color='red', linewidth=2, label='Regression Line')

plt.xlabel('Petal Length (cm)')
plt.ylabel('Sepal Length (cm)')
plt.title('Linear Regression (Scratch): Sepal Length vs. Petal Length')
plt.legend()
plt.grid(True)
plt.show()
```


Linear Regression (Scratch): Sepal Length vs. Petal Length
Actual Values
Regression Line
Sepal Length (cm)
Petal Length (cm)
7.0
6.5
6.0
5.5
5.0
4.5
1
2
3
4
5
6
7